

Example 1: Passing One Argument to a Thread

Scenario:

A **taxi service** is picking up customers. Each **taxi (thread)** picks up a **customer (passed as an argument)**.

C++ Code:

```
/* Example 1: Passing One Argument to a Thread (Using pthread_join) */
#include <iostream>
#include <pthread.h>
#include <unistd.h>

void* taxiPickup(void* arg) {
    int customerID = *(int*)arg;
    std::cout << "Taxi is picking up Customer #" << customerID <<
"...\\n";
    sleep(2);
    std::cout << "Customer #" << customerID << " has been picked
up!\\n";
    pthread_exit(NULL);
}

int main() {
    pthread_t taxis[3];
    int customerIDs[3] = {101, 102, 103};

    for (int i = 0; i < 3; i++) {
        pthread_create(&taxis[i], NULL, taxiPickup, &customerIDs[i]);
        pthread_join(taxis[i], NULL); // Ensure sequential execution
    }

    std::cout << "All customers have been picked up!\\n";
    return 0;
}
```

What's Happening in the Code?

- Each taxi thread receives a customer ID as an argument.
 - The taxi picks up the customer, simulating a delay using `sleep(2)`.
 - `pthread_join` ensures all taxis complete their pickups before ending the program.
-

Example 2: Passing Multiple Arguments to a Thread

Scenario:

A food delivery service has multiple drivers (threads) delivering orders to different customers.

Each order has:

- Order ID
- Customer Name

We pass these two arguments using a struct.

C++ Code:

```
/* Example 2: Passing Multiple Arguments to a Thread (Using
pthread_join) */
#include <iostream>
#include <pthread.h>
#include <unistd.h>

struct Order {
    int orderID;
```

```

        std::string customerName;
    };

void* deliverOrder(void* arg) {
    Order* orderDetails = (Order*)arg;
    std::cout << "Delivering Order #" << orderDetails->orderID << " to "
    << orderDetails->customerName << "...\\n";
    sleep(2);
    std::cout << "Order #" << orderDetails->orderID << " delivered to "
    << orderDetails->customerName << "!\\n";
    pthread_exit(NULL);
}

int main() {
    pthread_t drivers[3];
    Order orders[3] = { {301, "Ali"}, {302, "Sara"}, {303, "Ahmed"} };

    for (int i = 0; i < 3; i++) {
        pthread_create(&drivers[i], NULL, deliverOrder, &orders[i]);
        pthread_join(drivers[i], NULL); // Ensure sequential execution
    }

    std::cout << "All orders have been delivered!\\n";
    return 0;
}

```

What's Happening in the Code?

- A struct (**Order**) is used to pass multiple arguments (Order ID & Customer Name).
 - Each thread receives a unique order and delivers it.
 - **pthread_join** ensures all deliveries are completed before exiting the program.
-

Example 3: Synchronizing Multiple Threads Using `pthread_join`

Scenario:

A school has three students (threads) taking an exam.

Each student takes a different amount of time to complete the test.

The exam supervisor (main thread) waits until all students finish before announcing,

"Exam is over!"

C++ Code:

```
/* Example 3: Synchronizing Multiple Threads Using pthread_join */
#include <iostream>
#include <pthread.h>
#include <unistd.h>

void* takeExam(void* arg) {
    int studentID = *(int*)arg;
    std::cout << "Student #" << studentID << " is taking the
exam...\n";
    sleep(2 + studentID);
    std::cout << "Student #" << studentID << " has finished the
exam!\n";
    pthread_exit(NULL);
}

int main() {
    pthread_t students[3];
    int studentIDs[3] = {1, 2, 3};

    for (int i = 0; i < 3; i++) {
        pthread_create(&students[i], NULL, takeExam, &studentIDs[i]);
    }

    for (int i = 0; i < 3; i++) {
        pthread_join(students[i], NULL); // Ensures all students
complete before proceeding
    }

    std::cout << "Exam is over! All students have finished!\n";
}
```

```
return 0;
}
```

✓ What's Happening in the Code?

- Each student thread receives an **ID** and simulates an exam.
 - Different students finish at different times (**sleep(2 + studentID)**).
 - The main program (**supervisor**) waits (**pthread_join**) until all students finish the exam.
 - After all students are done, the supervisor announces **"Exam is over!"**
-

Final Summary

pthread_create() starts a new thread.

Arguments are passed using **void*** and typecast back inside the function.

pthread_join() ensures the main thread waits for all threads to finish.

struct is useful for passing multiple arguments